# Sprawozdanie z laboratorium: Podstawy Programowania

Kalkulator Odwrotnej Notacji Polskiej (ONP) na mikrokontroler 80C552

Karolina Grodź, Kajetan Powała-Mosinski

10 listopada 2026

## 1 Cel ćwiczenia

Zadaniem laboratoryjnym było zaprojektowanie i napisanie w języku asembler (rodzina MCS-51) kalkulatora działającego w Odwrotnej Notacji Polskiej (ONP). Program miał obsługiwać liczby całkowite z zakresu 0-255, podstawowe operatory arytmetyczne (`+`, `-`, `*`, `/`) oddzielane spacją oraz klawisz `Enter` do obliczania wyniku. Dodatkowo dodano implementację obsługi błędów, w tym: braku argumentów, przekroczenia zakresu, dzielenia przez zero, przepełnienia stosu oraz wpisania nieznanego znaku.

## 2 Ogólny opis środowiska

Program uruchamiany jest na pakiecie uruchomieniowym wyposażonym w mikrokontroler 80C552. Do komunikacji z użytkownikiem program wykorzystuje interfejs szeregowy w standardzie RS232 oraz terminal komputerowy (np. Tera Term). Ponieważ na płytce znajduje się zintegrowany system operacyjny w pamięci EPROM, oprogramowanie korzysta z gotowych procedur systemowych (API) udostępnionych pod konkretnymi adresami w pamięci, takich jak `cin` (0x7F - odczyt znaku), `cout` (0x7D - wypisanie znaku) oraz `newline` (0x95).

## 3 Analiza najważniejszych elementów programu

Ze względu na specyfikę notacji ONP, kluczowym elementem programu było stworzenie niezależnego, programowego stosu dla obliczeń oraz implementacja logiki bezpośrednio na rejestrach mikrokontrolera.

- **Obsługa dedykowanego stosu kalkulatora** – Zamiast używać wbudowanego stosu procesora (`SP`), który obsługuje skoki i powroty z podprogramów, zdefiniowano osobny bufor w pamięci RAM (od adresu `40H`). Funkcje `push_a` i `pop_a` korzystają ze wskaźnika `wsk`, adresowania pośredniego (`@R0`) i ustawiają flagę przeniesienia `C` w przypadku przepełnienia.

```
1 push_a:
2     mov R1,A
3     mov A,wsk
4     cjne A,#buf_kon,push_ok
5     setb C                  ; flaga C=1 gdy stos jest pelny
6     ret
7 push_ok:
8     mov R0,A
9     mov A,R1
```

```
    mov @R0,A               ; zapis pod adres ze wskaznika
    inc wsk
    clr C
    ret
```

- **Operacje arytmetyczne i detekcja błędów (np. dzielenie)** – Każdy operator pobiera z własnego stosu dwa argumenty (lewy do `R4`, prawy do `R5`). Poniżej przedstawiono implementację dzielenia, która dodatkowo weryfikuje próbę dzielenia przez 0 przed wykonaniem instrukcji `div AB`.

```
op_dziel:
    cjne A,#2FH,blad_znak  ; jesli to nie znak '/', rzuc blad
    lcall pobierz
    jc blad_arg            ; brak wymaganych dwoch argumentow
    mov A,R5
    jz blad_zero           ; sprawdzenie: dzielenie przez 0
    mov A,R4
    mov B,R5
    div AB                 ; iloraz zapisywany w A
    sjmp odloz             ; odlozenie wyniku na stos
```

- **Konwersja z systemu binarnego na dziesiętny (ASCII)** – Wynik wszystkich operacji mieści się w jednym rejestrze (0–255). Aby wyświetlić go na ekranie terminala, program musi wyodrębnić setki, dziesiątki i jedności poprzez wielokrotne dzielenie przez 100 i 10.

```
wynik:
    mov B,#100
    div AB                 ; A = setki, B = reszta
    mov R6,A
    mov A,B
    mov B,#10
    div AB                 ; A = dziesiatki, B = jednosci
    mov R5,A
    mov R4,B
    ; [...] W dalszej czesci kodu program dodaje offset #30H
    ; do kazdej cyfry (konwersja na ASCII) i wygasza zera wiodace
```

- **Wczytywanie liczb wielocyfrowych** – Gdy program odbiera ciąg znaków, cyfry muszą być łączone w jedną liczbę algorytmem $nowa_wartosc = (stara_wartosc \times 10) + nowa_cyfra$. Program sprawdza po drodze, czy iloczyn nie wykracza poza rozmiar rejestru 8-bitowego.

```
    mov A,R3               ; R3 - wczytywana do tej pory liczba
    mov B,#10
    mul AB
    xch A,B                ; sprawdzenie starszego bajtu iloczynu
    jnz cyf_blad           ; jesli B!=0, wynik > 255
    xch A,B
    add A,R6               ; dodanie nowo wczytanej cyfry (z R6)
    jc cyf_blad
    mov R3,A
```

# 4 Wnioski

Opracowanie kalkulatora w Odwrotnej Notacji Polskiej w środowisku asemblera mikrokontrolera 8051 wymaga precyzyjnego zarządzania pamięcią. Stworzenie dedykowanego, programowego bufora jako stosu uchroniło program przed konfliktami ze stosem sprzętowym wykorzystywanym przez podprogramy. Skrupulatna implementacja mechanizmów detekcji błędów na podstawie flag procesora (`C`, `Z`) gwarantuje stabilność kalkulatora, poprawnie wychwytując sytuacje takie jak nadmiar liczbowy, dzielenie przez zero czy niepoprawną formułę wejściową.